



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorio de docencia

Laboratorios de computación salas A y B

Profesor(a): Ing. Karina García Morales

Asignatura: Fundamentos de Programación (L)

Grupo: 22

No. de práctica(s): Práctica 10: Arreglos multidimensionales

Integrante(s): González Márquez José Luis

No. de lista o brigada: 20

Semestre: 2026-1

Fecha de entrega: Martes 11 de noviembre de 2025

Observaciones:

CALIFICACIÓN: _____

Objetivo: El alumno utilizará arreglos de dos dimensiones en la elaboración de programas que resuelvan problemas que requieran agrupar datos del mismo tipo, en estructuras que utilicen dos índices

Desarrollo:

Arreglos Multidimensionales en C : Un arreglo multidimensional en C es, en esencia, un "arreglo de arreglos". Permite almacenar datos del mismo tipo en una estructura de múltiples dimensiones, siendo los más comunes los de dos dimensiones (bidimensionales), conocidos como matrices o tablas.

`tipoDato nombre[tam1][tam2];` // Sintaxis para un arreglo 2D (matriz)

Almacenamiento en Memoria : Aunque lógicamente los vemos como una cuadrícula (filas y columnas), en la memoria del sistema se almacenan en un bloque único y contiguo. C organiza los elementos por filas (orden "row-major"), poniendo todos los elementos de la primera fila, seguidos por todos los de la segunda, y así sucesivamente.

Métodos de Acceso y Recorrido

Existen dos formas principales de acceder a los elementos de una matriz:

1. Usando Bucles Anidados (El método intuitivo)

Es la forma más común. Se utiliza un bucle por cada dimensión; para una matriz 2D, se usan dos bucles anidados (uno para las filas *i* y otro para las columnas *j*). Esto funciona con `for`, `while` o `do-while`.

Acceso: Se usa la sintaxis de índices `matriz[i][j]`.

Ventaja: Es muy claro y fácil de leer, ya que respeta la estructura lógica de filas y columnas.

2. Usando Apuntadores (El método eficiente)

Dado que la matriz es contigua en memoria, se puede tratar como un solo arreglo unidimensional gigante (de tamaño `filas * columnas`).

Acceso: Se declara un apuntador al inicio (`int *ap = &matriz[0][0];`) y se recorre con un solo bucle, accediendo a los elementos con aritmética de apuntadores (`*(ap + k)`).

Ventaja: Es más rápido y eficiente en el manejo de memoria, aunque puede ser menos intuitivo de leer.



Figura 1. Almacenamiento en memoria de un arreglo de 3X2

Actividades:

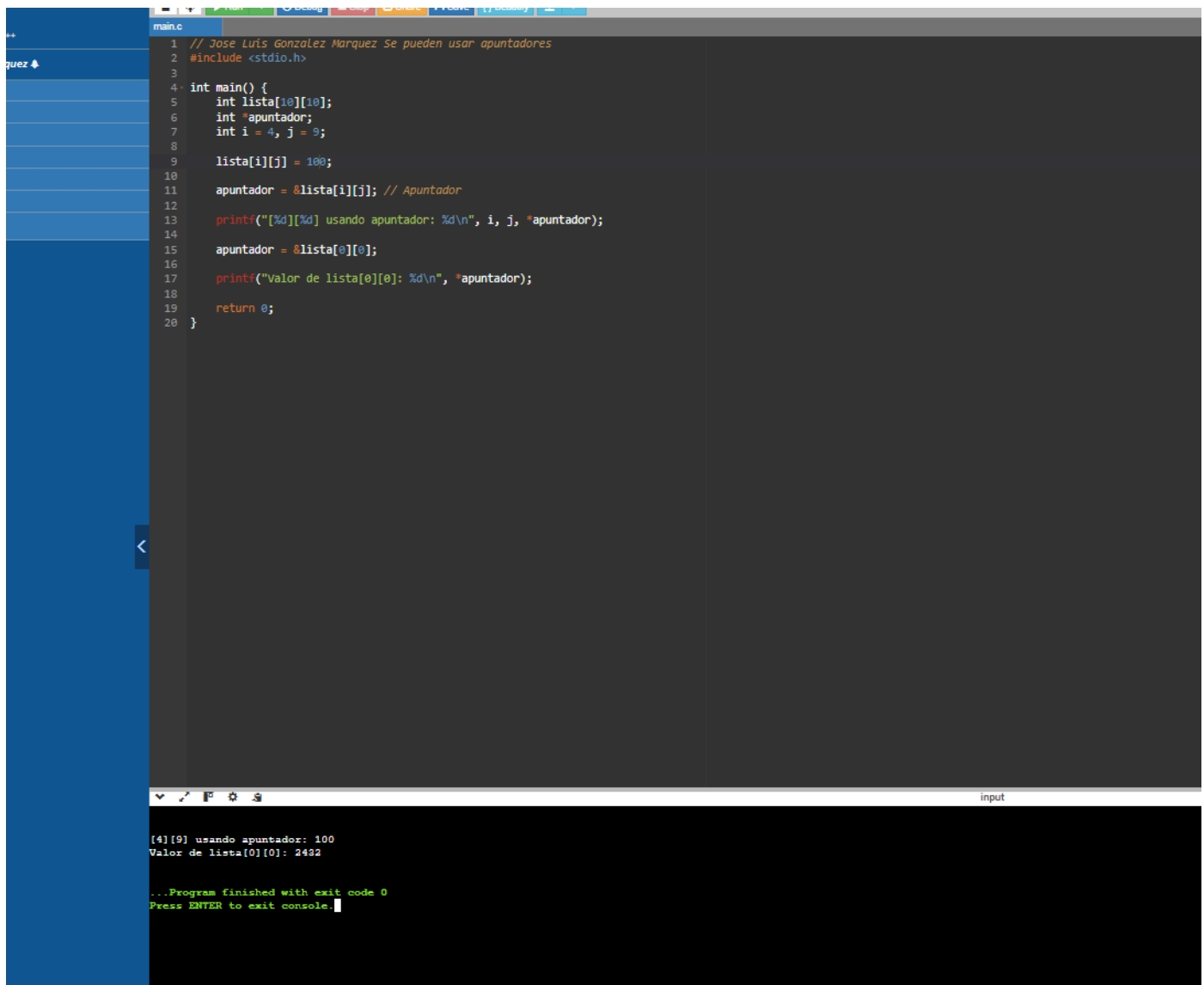
¿Qué es un arreglo bidimensional?

Un arreglo bidimensional (a menudo llamado matriz) es una estructura de datos que organiza elementos en una cuadrícula o tabla, utilizando filas y columnas. En lugar de tener una sola "fila" de datos (como un arreglo normal, o *unidimensional*), un arreglo bidimensional es, en esencia, un arreglo que contiene otros arreglos.

Realiza un ejemplo del uso de apuntadores en un arreglo bidimensional

```
int lista[10][10], *apuntador;
```

```
apuntador= &lista[i][j]; apuntador=*lista;
```

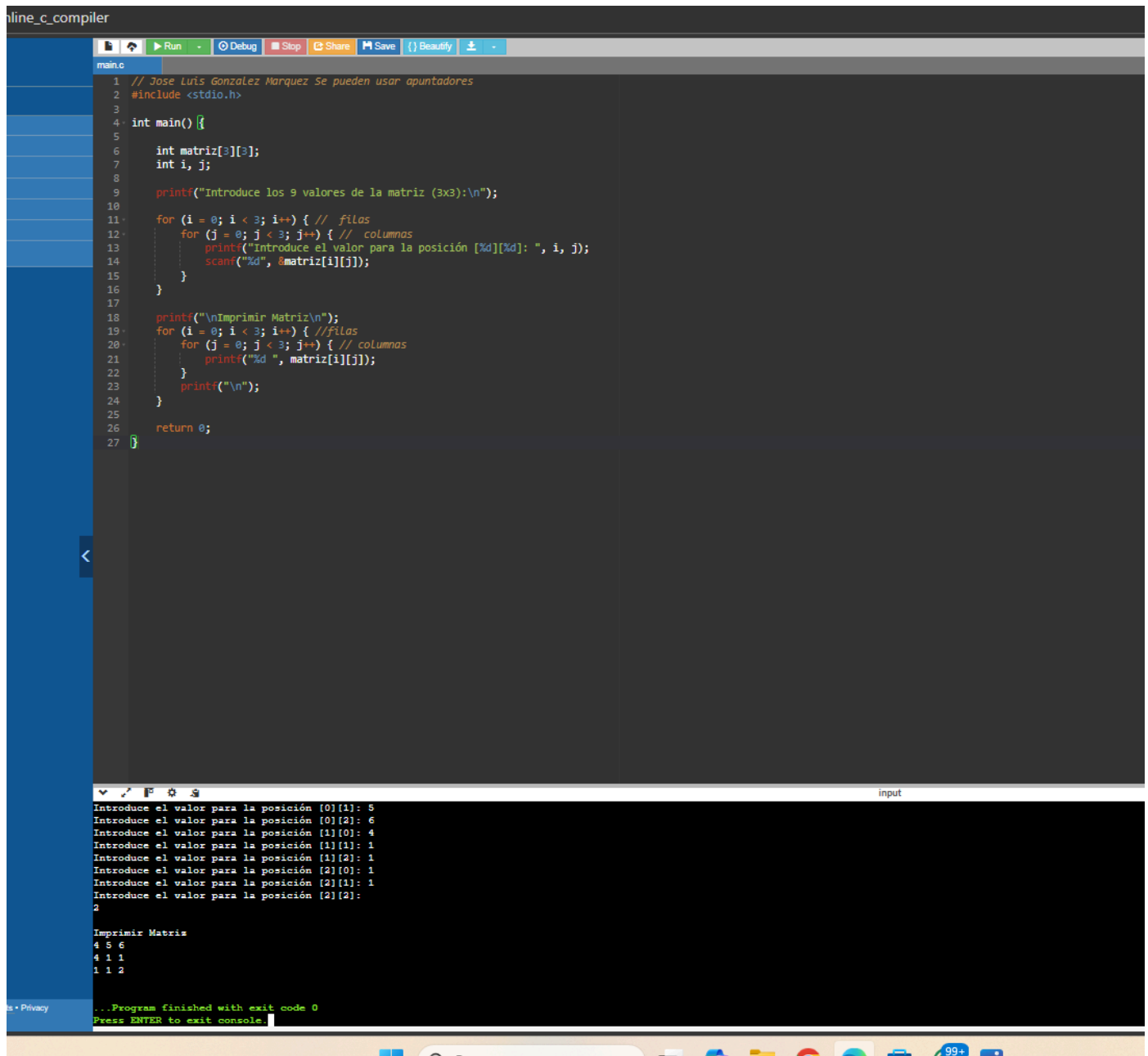
A screenshot of a code editor showing a C program. The code defines a 10x10 integer array 'lista', sets 'i' to 4 and 'j' to 9, assigns 'apuntador' the address of 'lista[i][j]', and prints this address. Then, it dereferences 'apuntador' to get the value at 'lista[0][0]' and prints it. The output shows the address 2442 and the value 100. The program ends with 'exit code 0'.

```
1 // Jose Luis Gonzalez Marquez Se pueden usar apuntadores
2 #include <stdio.h>
3
4 int main() {
5     int lista[10][10];
6     int *apuntador;
7     int i = 4, j = 9;
8
9     lista[i][j] = 100;
10
11     apuntador = &lista[i][j]; // Apuntador
12
13     printf("[%d][%d] usando apuntador: %d\n", i, j, *apuntador);
14
15     apuntador = &lista[0][0];
16
17     printf("Valor de lista[0][0]: %d\n", *apuntador);
18
19     return 0;
20 }
```

[4][9] usando apuntador: 100
Valor de lista[0][0]: 2442

...Program finished with exit code 0
Press ENTER to exit console.

1.- Del programa 1a.c modificar para que se pidan los datos por el usuario, de la matriz de 3 por 3



The screenshot shows an online C compiler interface. The top section displays the source code for a program that creates a 3x3 integer matrix, prompts the user to enter values for each element, and then prints the matrix. The code uses nested loops for both input and output. The bottom section shows the program's execution, where the user has entered values for each of the nine positions in the matrix. The output shows the matrix being printed as three rows of three numbers each. The program finished with an exit code of 0.

```
1 // Jose Luis Gonzalez Marquez Se pueden usar apuntadores
2 #include <stdio.h>
3
4 int main() {
5
6     int matriz[3][3];
7     int i, j;
8
9     printf("Introduce los 9 valores de la matriz (3x3):\n");
10
11     for (i = 0; i < 3; i++) { // filas
12         for (j = 0; j < 3; j++) { // columnas
13             printf("Introduce el valor para la posición [%d][%d]: ", i, j);
14             scanf("%d", &matriz[i][j]);
15         }
16     }
17
18     printf("\nImprimir Matriz\n");
19     for (i = 0; i < 3; i++) { //filas
20         for (j = 0; j < 3; j++) { // columnas
21             printf("%d ", matriz[i][j]);
22         }
23         printf("\n");
24     }
25
26     return 0;
27 }
```

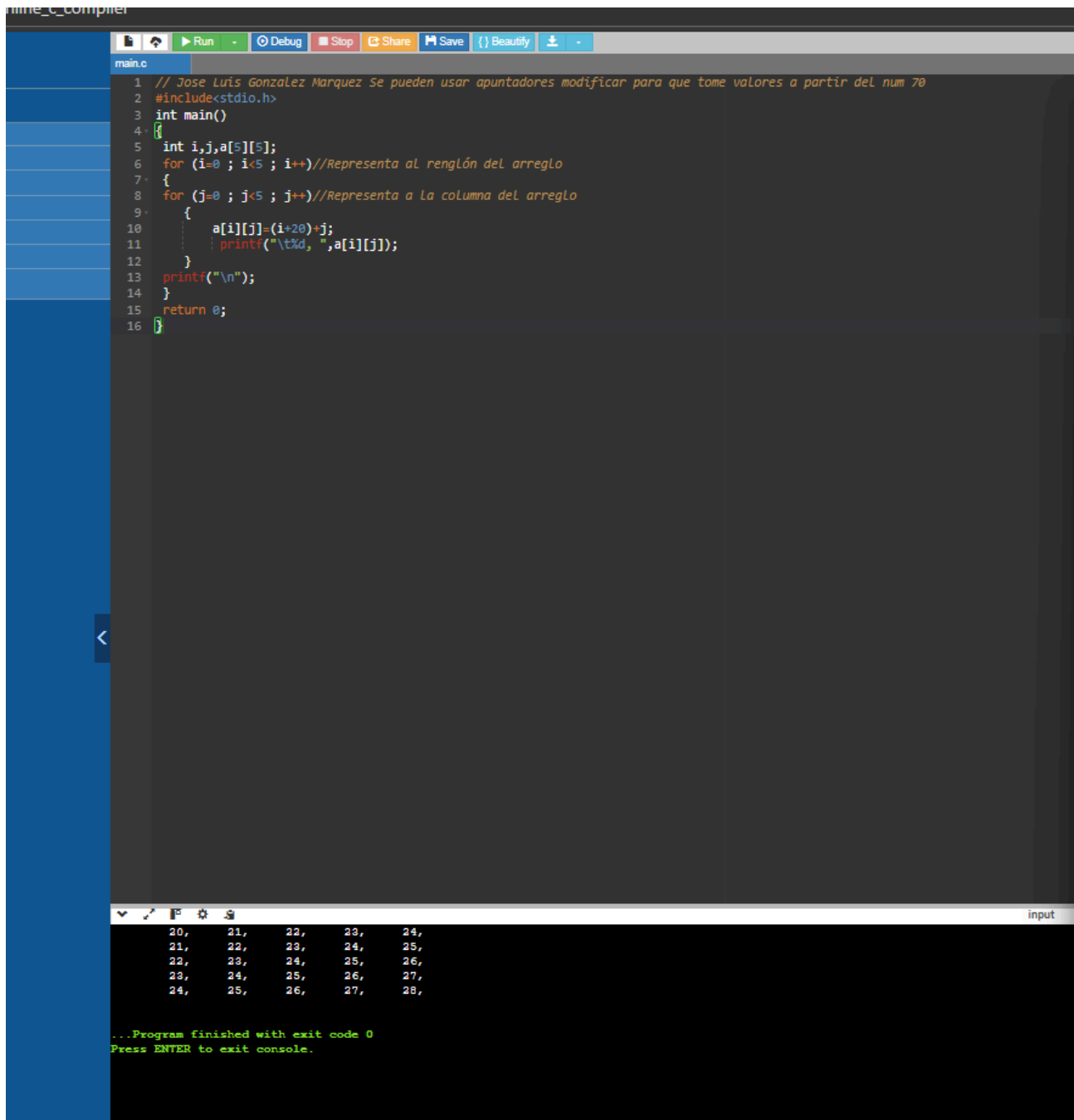
Introduce el valor para la posición [0][1]: 5
Introduce el valor para la posición [0][2]: 6
Introduce el valor para la posición [1][0]: 4
Introduce el valor para la posición [1][1]: 1
Introduce el valor para la posición [1][2]: 1
Introduce el valor para la posición [2][0]: 1
Introduce el valor para la posición [2][1]: 1
Introduce el valor para la posición [2][2]: 2

Imprimir Matriz
4 5 6
4 1 1
1 1 2

...Program finished with exit code 0
Press ENTER to exit console.

Solamente agregamos un printf y un scanf para que pudiera ver los valores que le pedimos al usuario

2.- Del programa 2a.c, modificar para que tome los valores a partir del numero 20, pueden utilizar diferentes formas



The screenshot shows an online C compiler interface with a file named 'main.c'. The code is as follows:

```
1 // Jose Luis Gonzalez Marquez Se pueden usar apuntadores modificar para que tome valores a partir del num 20
2 #include<stdio.h>
3 int main()
4 {
5     int i,j,a[5][5];
6     for (i=0 ; i<5 ; i++)//Representa al renglón del arreglo
7     {
8         for (j=0 ; j<5 ; j++)//Representa a la columna del arreglo
9         {
10             a[i][j]=(i+20)+j;
11             printf("\t%d, ",a[i][j]);
12         }
13         printf("\n");
14     }
15     return 0;
16 }
```

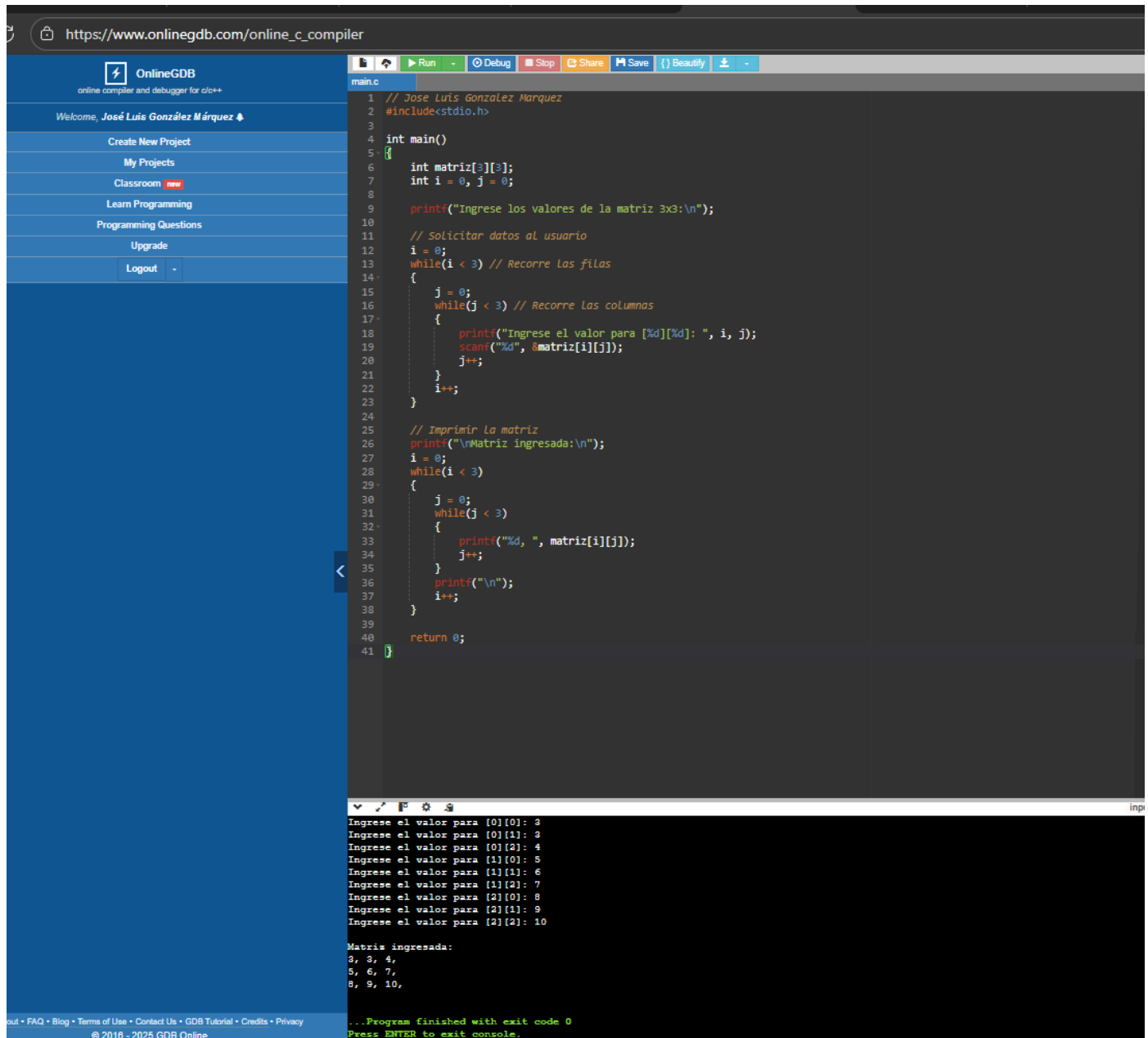
The output of the program is a 5x5 grid of numbers:

20,	21,	22,	23,	24,
21,	22,	23,	24,	25,
22,	23,	24,	25,	26,
23,	24,	25,	26,	27,
24,	25,	26,	27,	28,

Below the output, the console shows: "...Program finished with exit code 0" and "Press ENTER to exit console."

Aquí lo único que hicimos fue ponerle un +20

3.- Del programa 1b.c modificar para que solicite los datos al usuario con estructuras while



The screenshot shows the OnlineGDB website interface. The left sidebar contains navigation links: "OnlineGDB", "Welcome, José Luis González Márquez", "Create New Project", "My Projects", "Classroom", "Learn Programming", "Programming Questions", "Upgrade", and "Logout". The main area displays a C program named "main.c" with the following code:

```
1 // Jose Luis Gonzalez Marquez
2 #include<stdio.h>
3
4 int main()
5 {
6     int matriz[3][3];
7     int i = 0, j = 0;
8
9     printf("Ingrese los valores de la matriz 3x3:\n");
10
11     // Solicitar datos al usuario
12     i = 0;
13     while(i < 3) // Recorre Las filas
14     {
15         j = 0;
16         while(j < 3) // Recorre Las columnas
17         {
18             printf("Ingrese el valor para [%d][%d]: ", i, j);
19             scanf("%d", &matriz[i][j]);
20             j++;
21         }
22         i++;
23     }
24
25     // Imprimir La matriz
26     printf("\nMatriz ingresada:\n");
27     i = 0;
28     while(i < 3)
29     {
30         j = 0;
31         while(j < 3)
32         {
33             printf("%d, ", matriz[i][j]);
34             j++;
35         }
36         printf("\n");
37         i++;
38     }
39
40     return 0;
41 }
```

The output window at the bottom shows the program's execution results:

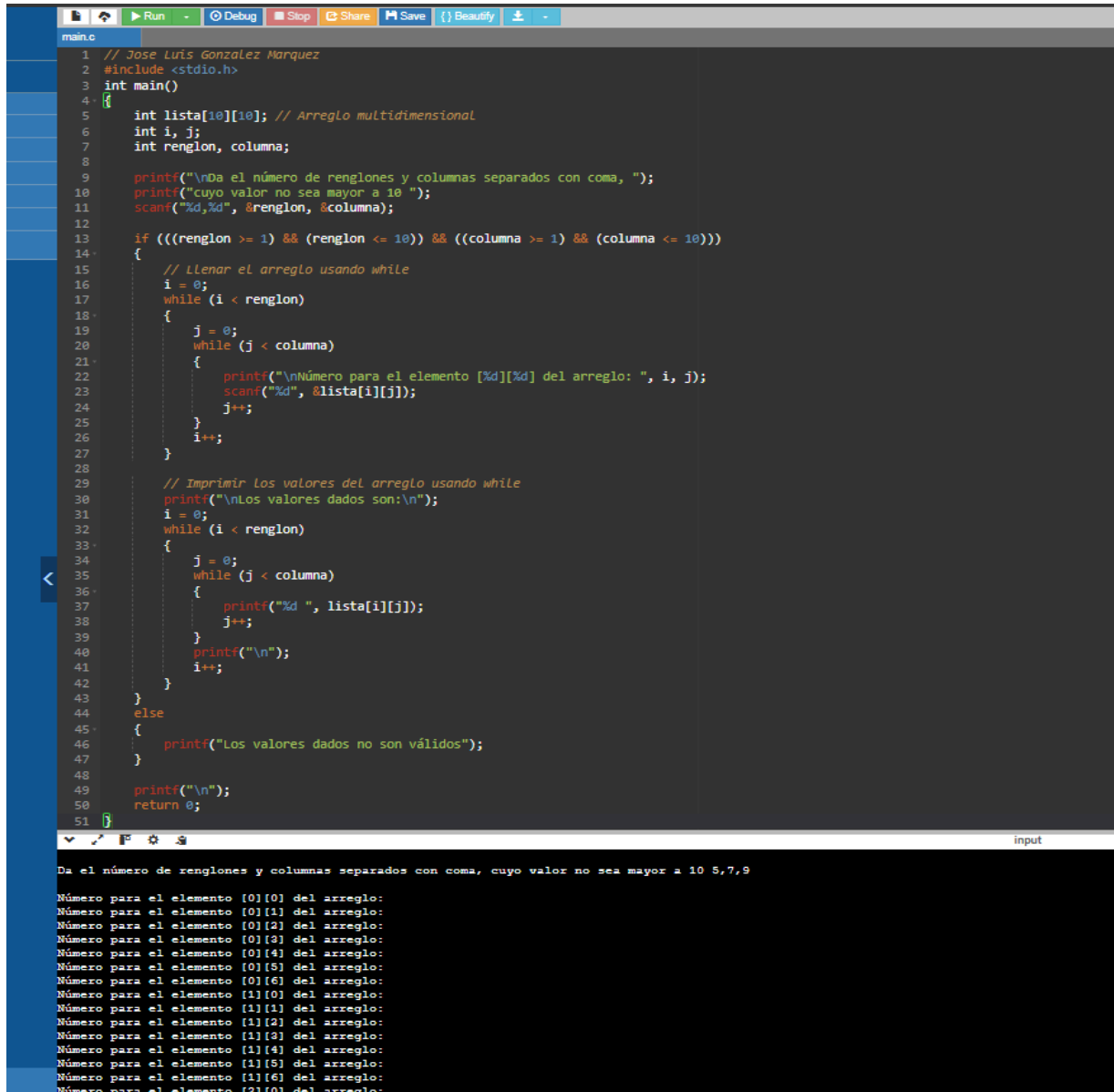
```
Ingrese el valor para [0][0]: 3
Ingrese el valor para [0][1]: 3
Ingrese el valor para [0][2]: 4
Ingrese el valor para [1][0]: 5
Ingrese el valor para [1][1]: 6
Ingrese el valor para [1][2]: 7
Ingrese el valor para [2][0]: 8
Ingrese el valor para [2][1]: 9
Ingrese el valor para [2][2]: 10

Matriz ingresada:
3, 3, 4,
5, 6, 7,
8, 9, 10,

...Program finished with exit code 0
Press ENTER to exit console.
```

Aqui usamos ciclos while anidados. Para poder trabajar mejor

4.- Del programa 3.c van a indicarme que realiza el programa y van a modificar todas las estructuras for sustituyendo por while o do-while



```
main.c
1 // Jose Luis Gonzalez Marquez
2 #include <stdio.h>
3 int main()
4 {
5     int lista[10][10]; // Arreglo multidimensional
6     int i, j;
7     int renglon, columna;
8
9     printf("\nDa el número de renglones y columnas separados con coma, ");
10    printf("cuyo valor no sea mayor a 10 ");
11    scanf("%d,%d", &renglon, &columna);
12
13    if (((renglon >= 1) && (renglon <= 10)) && ((columna >= 1) && (columna <= 10)))
14    {
15        // Llenar el arreglo usando while
16        i = 0;
17        while (i < renglon)
18        {
19            j = 0;
20            while (j < columna)
21            {
22                printf("\nNúmero para el elemento [%d][%d] del arreglo: ", i, j);
23                scanf("%d", &lista[i][j]);
24                j++;
25            }
26            i++;
27        }
28
29        // Imprimir los valores del arreglo usando while
30        printf("\nLos valores dados son:\n");
31        i = 0;
32        while (i < renglon)
33        {
34            j = 0;
35            while (j < columna)
36            {
37                printf("%d ", lista[i][j]);
38                j++;
39            }
40            printf("\n");
41            i++;
42        }
43    }
44    else
45    {
46        printf("Los valores dados no son válidos");
47    }
48
49    printf("\n");
50    return 0;
51 }
```

input

Da el número de renglones y columnas separados con coma, cuyo valor no sea mayor a 10 5,7,9

Número para el elemento [0][0] del arreglo:
Número para el elemento [0][1] del arreglo:
Número para el elemento [0][2] del arreglo:
Número para el elemento [0][3] del arreglo:
Número para el elemento [0][4] del arreglo:
Número para el elemento [0][5] del arreglo:
Número para el elemento [0][6] del arreglo:
Número para el elemento [1][0] del arreglo:
Número para el elemento [1][1] del arreglo:
Número para el elemento [1][2] del arreglo:
Número para el elemento [1][3] del arreglo:
Número para el elemento [1][4] del arreglo:
Número para el elemento [1][5] del arreglo:
Número para el elemento [1][6] del arreglo:
Número para el elemento [2][0] del arreglo:

Pues solamente solicita al usuario los valores para crear la matriz, y dependiendo de la multiplicación de éstos dos dependerá de los valores que se solicitarán
Para eso sirven los while, para que no pase los valores solicitados de 10.

5.- Del programa 3.c modificar aplicando apuntadores

```
main.c
1 // Jose Luis Gonzalez Marquez Se pueden usar apuntadores
2 #include <stdio.h>
3
4 int main ()
5 {
6     int lista[10][10];
7     int *apuntador;
8     int i,j;
9     int renglon,columna;
10
11     printf("\nDa el número de renglones y columnas separados con coma, ");
12     printf("cuyo valor no sea mayor a 10 ");
13     scanf("%d,%d",&renglon,&columna);
14
15     if(((renglon>=1) && (renglon<=10))&&((columna>=1) && (columna<=10)))
16     {
17
18         printf("\n Captura de datos (con apuntadores) \n");
19         for (i= 0 ; i <= renglon-1 ; i++)
20         {
21             for(j= 0 ; j <= columna-1 ; j++)
22             {
23                 printf("Número para el elemento [%d][%d] del arreglo: ", i,j );
24
25                 apuntador = &lista[i][j];
26
27                 scanf("%d", apuntador);
28             }
29         }
30
31         printf("\nLos valores dados son: \n");
32
33         for (i= 0 ; i <= renglon-1 ; i++)
34         {
35             for(j= 0 ; j <= columna-1 ; j++)
36             {
37
38                 apuntador = &lista[i][j];
39
40                 printf("%d ", *apuntador);
41             }
42             printf("\n");
43         }
44     }
45     else printf("Los valores dados no es válido");
46
47     printf("\n");
48     return 0;
49 }
```

input

```
Captura de datos (con apuntadores)
Número para el elemento [0][0] del arreglo: 1
Número para el elemento [0][1] del arreglo: 1
Número para el elemento [0][2] del arreglo: 1
Número para el elemento [0][3] del arreglo: 1
Número para el elemento [1][0] del arreglo: 1
Número para el elemento [1][1] del arreglo: 1
Número para el elemento [1][2] del arreglo: 1
Número para el elemento [1][3] del arreglo: 1

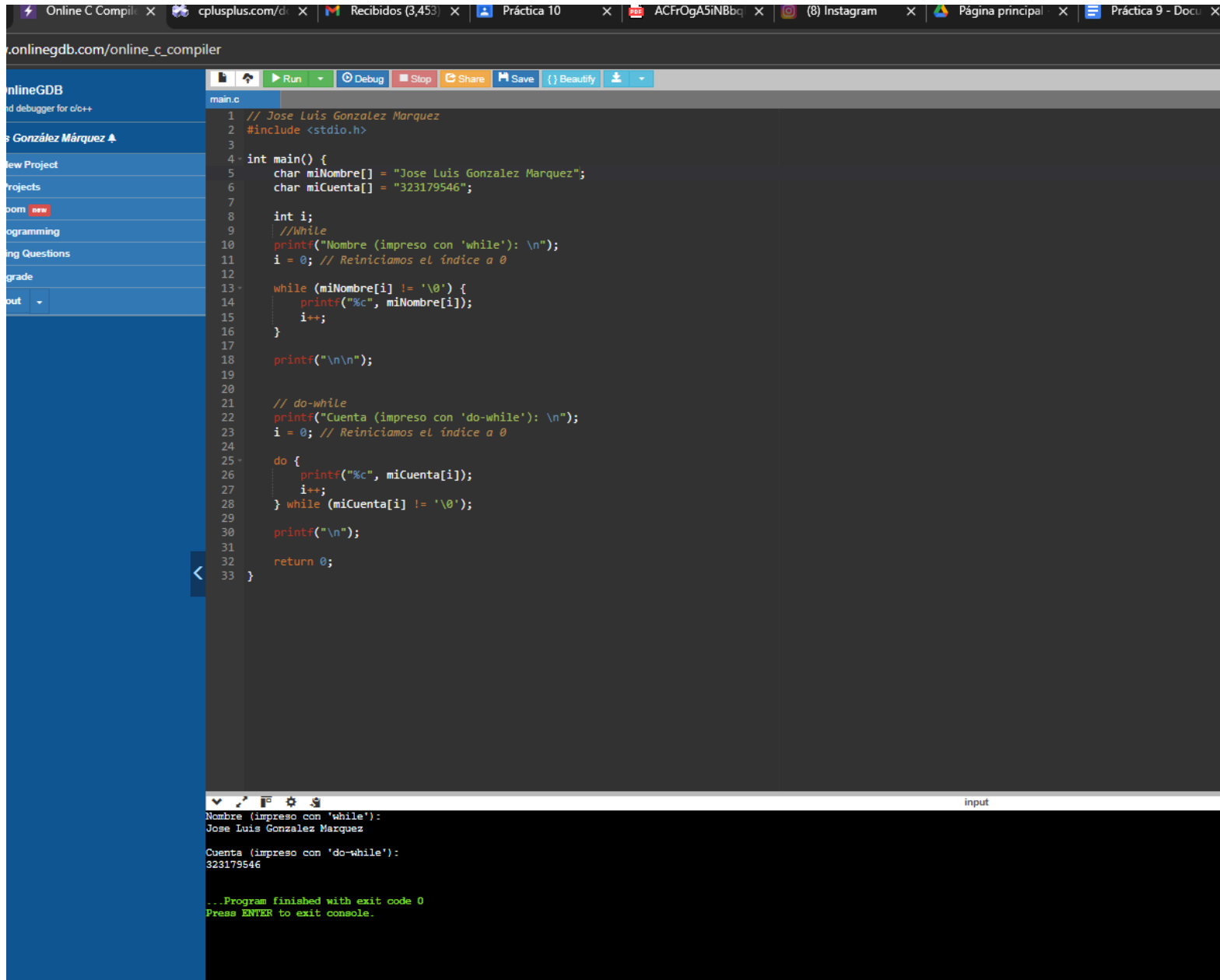
Los valores dados son:
1 1 1 1
1 1 1 1

...Program finished with exit code 0
Press ENTER to exit console.
```

Aquí utilizamos el apuntador para poder asignar y poder almacenar los datos. Con eso logramos la matriz pero igual poniendo columnas y filas.

Tarea:.

1.-Realiza un programa que muestre tu nombre y número de cuenta haciendo uso de 2 arreglos, emplear while y do-while



The screenshot shows a web browser with several tabs open, including 'Online C Compiler', 'cplusplus.com', 'Recibidos (3,453)', 'Práctica 10', 'ACFrOgASiN8bq', '(8) Instagram', 'Página principal', and 'Práctica 9 - Docu'. The main content area displays the 'onlinegdb.com/online_c_compiler' interface. On the left is a sidebar with navigation links like 'onlineGDB', 'New Project', 'Projects', 'pom', 'Programming', 'ing Questions', 'grade', and 'out'. The main editor shows a C program in 'main.c' with the following code:

```
1 // Jose Luis Gonzalez Marquez
2 #include <stdio.h>
3
4 int main() {
5     char miNombre[] = "Jose Luis Gonzalez Marquez";
6     char miCuenta[] = "323179546";
7
8     int i;
9     //while
10    printf("Nombre (impreso con 'while'): \n");
11    i = 0; // Reiniciamos el indice a 0
12
13    while (miNombre[i] != '\0') {
14        printf("%c", miNombre[i]);
15        i++;
16    }
17
18    printf("\n\n");
19
20    // do-while
21    printf("Cuenta (impreso con 'do-while'): \n");
22    i = 0; // Reiniciamos el indice a 0
23
24    do {
25        printf("%c", miCuenta[i]);
26        i++;
27    } while (miCuenta[i] != '\0');
28
29    printf("\n");
30
31    return 0;
32 }
33 }
```

At the bottom, the console output shows the program's execution:

```
Nombre (impreso con 'while'):
Jose Luis Gonzalez Marquez

Cuenta (impreso con 'do-while'):
323179546

...Program finished with exit code 0
Press ENTER to exit console.
```

2.- Corrige e indica que realiza el siguiente programa:(Encuentra los 5 errores)

```
#include<studio.h>
```

```
main()
```

```
{
```

```
    int fila,columna, cont=0,contadorApuntador;
```

```
    float arreglo[3][3], acumulador=0, *ap;
```

```
    ap = &arreglo[0][0];
```

```
    for(fila=0; fila<3; fila++)
```

```
    {
```

```
        for(columna=0; columna<1;columna--)
```

```
        {
```

```
            printf("\t\n Teclear el elemento %d %d \t", fila+1,columna+1);
```

```
            scanf("%f",&arreglo[fila][columna]);
```

```
            acumulador+=arreglo[fila][columna];
```

```
        }
```

```
}
```

```
printf("\tLa Matriz es:\n");
```

```
for (contadorApuntador=0 ; contadorApuntador<9 ; contadorApuntador++)
```

```
{
```

```
    if (cont == 3)
```

```
    {
```

```
        printf("\n");
```

```
        cont = 1;
```

```
    }
```

```
    printf("%.2f\t\t",*(ap+contadorApuntador));
```

```
    cont++;
```

```
}
```

```
printf("\t\n\n La suma de los elementos es:%0.2d", acumulador);
```

```
return 0;
```

}

Los 5 errores y cómo corregirlos:

- 1-El primer error del que me di cuenta fue de que printf y scanf está mal escrito.
- 2-Después el segundo error fue que el for interno está mal y eso hace un bucle infinito
- 3-El error aquí fue que al final, intentas imprimir la variable pero como no es entero nos da error.
- 4-Aquí al imprimir la matriz, el contador cont se reinicia a 1 en lugar de 0.
- 5-Por último fue que requiere que la función main especifique un tipo de retorno, que casi siempre es int.

¿Qué hace?

El programa hace una matriz de 3x3 , también declara un puntero *ap y lo hace apuntar al primer elemento de la matriz. Después pide al usuario que ingrese los 9 números de la matriz, uno por uno, usando dos bucles for anidados y por último calcula la suma total de todos los elementos a medida que el usuario los introduce y la guarda en la variable acumulador.

```

1 // Jose Luis Gonzalez Marquez
2 #include <stdio.h> // Error 1 corregido
3
4 int main() // Aqui falta el int
5 {
6     int fila, columna, cont = 0, contadorApuntador;
7     float arreglo[3][3], acumulador = 0, *ap;
8
9     ap = &arreglo[0][0]; // este iba antes
10
11     for (fila = 0; fila < 3; fila++)
12     {
13         for (columna = 0; columna < 3; columna++) // era mas mas
14         {
15             printf("\t\n Teclear el elemento %d %d \t", fila + 1, columna + 1);
16             scanf("%f", &arreglo[fila][columna]);
17             acumulador += arreglo[fila][columna];
18         }
19     }
20
21     printf("\n\tLa Matriz es:\n");
22
23     for (contadorApuntador = 0; contadorApuntador < 9; contadorApuntador++)
24     {
25         if (cont == 3)
26         {
27             printf("\n");
28             cont = 0; // Era 0
29         }
30         printf("%.2f\t\t", *(ap + contadorApuntador));
31         cont++;
32     }
33
34     // %.2f para el float acumulador
35     printf("\t\n\tLa suma de los elementos es: %.2f", acumulador);
36
37     return 0;
38 }
39

```

```

Teclear el elemento 3 1
31
Teclear el elemento 3 2      32
Teclear el elemento 3 3      0

      La Matriz es:
11.00      12.00      13.00
21.00      22.00      23.00
31.00      32.00      0.00

La suma de los elementos es: 165.00

```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

3.- Programa que solicite al usuario los valores de dos matrices de 3 x 3 y haga su multiplicación haciendo uso de arreglos. (Investigar como se realiza una multiplicación de matrices)

```
online_c_compiler
main.c
1 // Jose Luis Gonzalez Marquez Se pueden usar apuntadores
2 #include <stdio.h>
3
4 int main() {
5     int A[3][3];
6     int B[3][3];
7     int C[3][3];
8     int i, j, k;
9
10    printf("--- Introduzca los valores de la Matriz A (3x3) ---\n");
11    for (i = 0; i < 3; i++) {
12        for (j = 0; j < 3; j++) {
13            printf("Elemento A[%d][%d]: ", i, j);
14            scanf("%d", &A[i][j]);
15        }
16    }
17    //Revisar aqui
18    printf("\n--- Introduzca los valores de la Matriz B (3x3) ---\n");
19    for (i = 0; i < 3; i++) {
20        for (j = 0; j < 3; j++) {
21            printf("Elemento B[%d][%d]: ", i, j);
22            scanf("%d", &B[i][j]);
23        }
24    }
25
26    printf("\nCalculando C = A * B ...\n");
27
28    for (i = 0; i < 3; i++) {
29        for (j = 0; j < 3; j++) {
30            C[i][j] = 0;
31
32            for (k = 0; k < 3; k++) {
33                C[i][j] += A[i][k] * B[k][j];
34            }
35        }
36    }
37
38    printf("\nMatriz Resultante C (A * B) \n");
39    for (i = 0; i < 3; i++) {
40        for (j = 0; j < 3; j++) {
41            printf("%d\t", C[i][j]);
42        }
43        printf("\n");
44    }
45    printf("\n");
46    return 0;
47 }
48
49
50
51
input
Elemento B[1][1]: 1
Elemento B[1][2]: 1
Elemento B[2][0]: 1
Elemento B[2][1]: 2
Elemento B[2][2]: 2
Calculando C = A * B ...
Matriz Resultante C (A * B)
4    6    6
9    11   11
26   48   48
...Program finished with exit code 0
Press ENTER to exit console.
```

Conclusión:

Sinceramente esta práctica se me dificultó más porque esta vez no vimos tanta teoría o conceptos que aprendernos, fue analizar mucho, hacer y muchas más cosas con los programas. Lo que me pareció gracioso fue que el código escrito no lo entendía muy bien, pero al pasarlo a la página para poderlo ejecutar fue cuando se me hizo más fácil entenderlo. Creo que el más difícil fue el dos ya que eran cinco errores del programa, pero aprendí bastante y espero me vaya mejor en la práctica once después de ver en dónde estaba fallando.

Fuentes:

Facultad de Ingeniería. (2025). Manual de prácticas del laboratorio de Fundamentos de Programación. Laboratorio de computación. Salas A y B. Universidad Nacional Autónoma de México. pp. 149-160. Recuperado el 11 de noviembre de 2025 de <http://lcp02.fi-b.unam.mx/>

Tutorial práctico (GeeksforGeeks): GeeksforGeeks. (2025, 25 de julio). *Multidimensional Arrays in C - 2D and 3D Arrays*. <https://www.geeksforgeeks.org/c/multidimensional-arrays-in-c/>